07 快速排序

Part II Sorting and Order Statistics

本章定位

Quicksort的分割只建立兩側與pivot的大小關係，還要遞迴才完成排序。本章從Lomuto不變量追到完整數值狀態，再把最壞時間、內部亂數的期望比較數與stack深度分開分析，特別處理重複鍵造成的退化。

前置知識

陣列分割、遞迴與期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 實作 partition 並維持正確的分割不變量。
- 區分最壞、最好與期望快速排序。
- 分析隨機選 pivot 如何解除對抗性輸入。
- 7.1 Description of quicksort
- 7.2 Performance of quicksort
- 7.3 A randomized version of quicksort
- 7.4 Analysis of quicksort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Quicksort先選pivot，將其他鍵按與pivot的大小分開，再遞迴排序兩側。partition完成時只保證區域間的大小關係，兩側內部尚未排序。本節採 1 起算索引、閉區間 A[p..r]，使用 Lomuto 分割，最後位置 r 的鍵作 pivot。

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空區間與單元素是基底；兩個遞迴子區間都排除q，所以嚴格縮小。若還把q放回同一側，遇pivot在端點就可能無限遞迴。

迴圈邊界：i 是小鍵區尾端，j 是下一筆待分類

每輪開始時，pivot 固定在 A[r]。本章採 **1 起算索引、閉區間 A[p..r]**；i 可以等於 p−1，表示 ≤ 區為空。

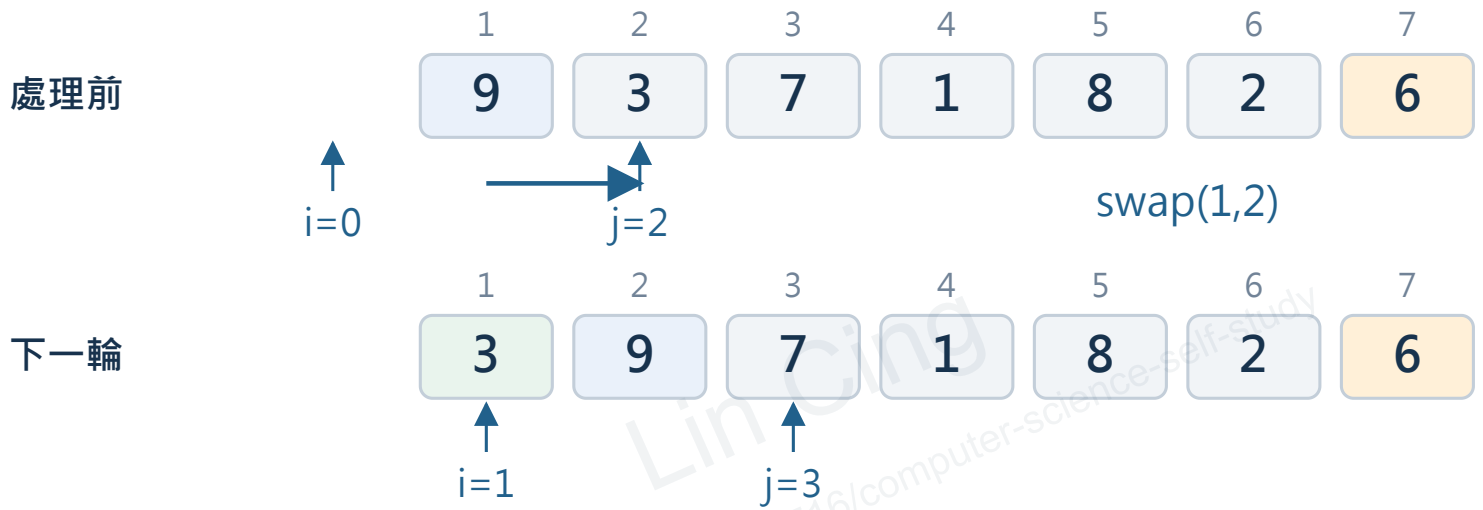
區域	已知關係	初始狀態 i=p−1、j=p
A[p..i]	≤ pivot	空
A[i+1..j−1]	> pivot	空
A[j..r−1]	尚未分類	除 pivot 外全部
A[r]	pivot	全程固定到收尾交換

讀圖方式： 綠色是 ≤ 區，藍色是 > 區，灰色是未知區，橘色是 pivot。以下兩頁都沿用 [9,3,7,1,8,2,6]、pivot=6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

小鍵到來：交換後，兩個邊界都前進

3 ≤ 6：先讓 i 前進，再交換 A[1] 與 A[2]

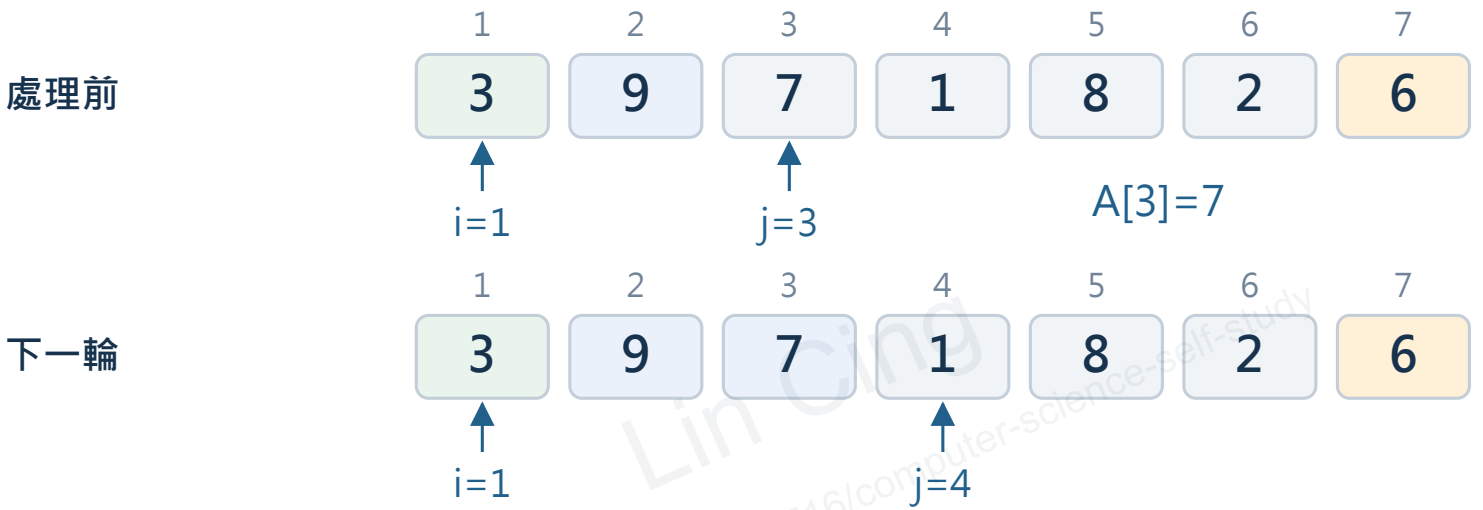


3 加入 ≤ 區；9 回到 > 區。pivot 6 仍在索引 7。

處理 A[2]=3 時，先令 i=1，再交換 A[1]、A[2]。3 加入 ≤ 區，原先的 9 留在 > 區；最後 j 前進。一般情況也可能 i=j，此時是自己與自己交換。

大鍵到來：不搬資料，只擴大已分類區

7 > 6：不交換、不改 i，只讓 j 前進



7 從未知區加入 > 區；陣列值不變，但分類邊界改變。

A[3]=7>6，因此 i 不變、陣列不變，只把 j 往右移一格。**不變的是值，改變的是已知資訊：** 7 現在屬於 > 區，不再是未知值。

收尾交換：q 才是 pivot 的最終位置



每輪未知區長 r−j 減一，所以掃描必終止。結束時 j=r；交換 A[i+1] 與 A[r]，得到 q=i+1。左區 ≤ pivot、右區 > pivot，兩側內部仍未排序。

從分割不變量到整體排序正確性

1. **初始化：** i=p−1、j=p，前兩區都是空區間，不變量成立。
2. **保持：** 小鍵交換後擴大 ≤ 區；大鍵只擴大 > 區。兩種情況都使未知區少一筆，pivot 仍固定。
3. **分割終止：** 未知區消失，收尾交換把 pivot 放到 q。交換只重排元素，不會遺失或新增資料。
4. **遞迴歸納：** 空區間、單元素已排序。兩個子區間排除 q，長度都變小；歸納假設使它們各自有序，再接上 pivot，整個區間便有序。

只檢查「左小右大」還不夠：完整正確性同時需要**元素多重集合不變、遞迴嚴格縮小**，以及兩側遞迴回來後確實有序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

完整數值分割追蹤

A=[9,3,7,1,8,2,6]，p=1、r=7、pivot=6，初i=0。

j	判斷	i更新	交換後陣列
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6]，swap1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6]，swap2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6]，swap3,6
結束	pivot放i+1	q=4	[3,1,2,6,8,7,9]，swap4,7

左[3,1,2]都≤6，右[8,7,9]都>6，內部並未排序。partition比較正好r−p次，時間Θ(r−p+1)、額外空間O(1)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最壞時間：一整族輸入達到平方成本

已排序的 n 個相異鍵，每次取最後位置作 pivot。pivot 永遠最大，子問題長度是 n−1 與 0。

本輪長度	本輪鍵比較	下一輪非空長度
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

比較總數為 $n(n-1)/2$，給出最壞時間的 $\Omega(n^2)$ 下界。這個結論適用整個輸入族，並非由單次實測推論。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

平方上界必須涵蓋每一種分割

令 C(n) 是 n 個鍵的最壞比較數，子問題大小 a+b=n−1。以較小規模的歸納假設可得

$$C(a)+C(b)+(n-1) \le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1 = \frac{n(n-1)}{2}-ab \le \frac{n(n-1)}{2}.$$

C(0)=C(1)=0，故上界涵蓋所有分割；a=0 或 b=0 時恰達界。每個被選作 pivot 的元素不再進入遞迴，所以呼叫總數 O(n)。總時間 O(n+C)，配上下界得到**最壞 $\Theta(n^2)$**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

均分時，先按層數清楚計算比較工作

取 n=15，每次選真正的中位數。此表只計 Lomuto 的鍵比較，單元素不呼叫 partition。

深度 d	子問題個數	每個長度	本層鍵比較
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

共 34 次；同樣 15 個鍵，端點分割則做 105 次。差別來自每層保留下來的子問題大小，與 partition 本身仍是線性掃描並不矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不必完全平分，固定比例也能保證對數深度

若每次非空子問題的較大側至多為原長度的 ρ 倍，其中 ρ<1 是固定常數，最長分支深度為 O(log n)。同一層的子區間互不重疊，本層總掃描量至多 n，因此總時間 O(n log n)；固定的 1:9 分割也符合。

完全均分且 $n = 2^k - 1$ 時，深度 d 有 2^d 個長度 $2^{k-d} - 1$ 的子問題。本層比較數為

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

前約 k/2 層，每層仍做 Θ(n) 次比較，故這種執行本身就需 Ω(n log n)，合為 Θ(n log n)。這裡的下界來自實際工作量；比較排序的最壞輸入下界並不保證每個輸入都一樣慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵如何破壞二路分割

對全相等A=[5,5,5,5,5]，Lomuto每次判斷≤都成立，q=r，左側長n−1、右側0。即使每輪先隨機選pivot，換來的仍是相同5，總比較仍10；一般n為n(n−1)/2。

因此下一節的隨機O(n log n)分析先假設鍵互異。若要支援大量重複鍵的效率，可使用小於/等於/大於三路partition，整段等值區不再遞迴；第9章會完整列出區間邊界與rank換算。

常見的「隨機pivot解決一切最壞輸入」缺少鍵相異或適當重複鍵處理的前提。穩定性也是不同性質，quicksort的長距離swap一般會改變相等鍵identity順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴空間與只遞迴較小側

原偽碼的兩次遞迴最壞可有n層堆疊，即使partition原地進行，整個quicksort也不是無條件O(1)額外空間。

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

每次真正的遞迴只選較小側，長度≤(n−1)/2；較大側由while繼續，不保留新一層frame。每層遞迴規模至少減半，故最壞堆疊O(log(n+1))。所有partition仍會執行，時間最壞仍$\Theta(n^2)$，不是藉減少stack就保證時間O(n log n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3 A randomized version of quicksort

對目前A[p..r]，從p..r均勻隨機選k，先swap(A[k],A[r])再用相同Lomuto partition。後續每次在當前子區間重新均勻選擇。鍵互異時，pivot的秩對1..n均勻，任何固定輸入排列都有同一個期望分析。

這是對演算法內部亂數取期望，不要求使用者資料的排列均勻。某次執行仍可能一直抽到端點而花 $\Theta(n^2)$；下面證明期望值，尚未把它當成每次執行的保證或未證的尾機率界。

理想模型採條件於目前狀態仍均勻的抽樣。固定且公開的偽亂數序列是可重現的實作選擇，其對抗性保證必須另外說明，不能把形式上的random呼叫當作數學均勻性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪兩個鍵會比較？看區間內誰先成為 pivot

將相異鍵按大小命名為 $z_1<\cdots<z_n$。$X_{ij}=1$ 表示 z_i、z_j 在某次 partition 直接比較。每對至多比較一次：直接比較時其中一個是 pivot，該 pivot 隨即退出後續子問題。

考慮秩區間 $Z_{ij}=\{z_i,\cdots,z_j\}$。區間外的 pivot 只能把整段留在同一側；在區間內第一個 pivot 出現前，z_i、z_j 始終同屬一個子問題。

分析排序後的秩：$Z_{2,5}$ = {2,3,4,5}；不是陣列快照



先選 7：2..5 一起留在左子問題，$X_{2,5}$ 尚未決定。

區間內先選端點 2（或 5）



該次比較 2 與 5 → $X_{2,5}=1$

區間內先選內部 3（或 4）



2 在左、5 在右 → 永不比較，$X_{2,5}=0$

四個區間鍵等可能先選；其中兩個是端點。$\Pr(X_{2,5}=1)=2/4=1/2$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

先選端點，與先選內部鍵，是互斥的兩種結果

以 z_2、z_5 為例，先忽略所有區間外的 pivot，只看 {2,3,4,5} 中誰最先被選中。

區間內第一個 pivot	2、5 的去向	$X_{2,5}$
2	本次 partition 將 2 與 5 直接比較	1
3	2 進左側，5 進右側；之後永不相遇	0
4	2 進左側，5 進右側；之後永不相遇	0
5	本次 partition 將 5 與 2 直接比較	1

每次在目前子區間均勻選 pivot，且選到區間外鍵不會區分這四個鍵，因此四者等可能成為區間內第一個 pivot。一般化後：

$$\Pr(X_{ij}=1)=\frac{1}{j-i+1}+\frac{1}{j-i+1}=\frac{2}{j-i+1}.$$

分母是區間長度 j−i+1，**不是整個輸入長度 n**。這個事件不要求該端點是全程第一個 pivot。

期望比較數：先按秩區間長度分組求和

總比較數 $C = \sum_{i<j} X_{ij}$。期望線性性不要求各事件獨立。令區間長 k=j−i+1；長度 k 的秩區間共有 n+1−k 個，所以

$$\begin{aligned} E[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)(H_n - 1) - 2(n-1) \\ &= 2(n+1)H_n - 4n, \quad H_n = \sum_{k=1}^{n} \frac{1}{k}. \end{aligned}$$

由 $\ln(n+1) \le H_n \le 1 + \ln n$ 得期望比較數 Θ(n log n)。迴圈工作與比較次數同階，加上 O(n) 次呼叫開銷，期望時間也是 Θ(n log n)。

這只使用期望線性性；比較事件彼此並不獨立，不能照搬獨立事件的變異數或尾機率公式。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同名 partition，邊界與成本模型可能不同

對照目前兩份教材的明定契約：

項目	本章：Lomuto	資料結構第 7 章：三路
索引與區間	1 起算、閉區間 A[p..r]	0 起算、右端不含 A[l:r]
回傳	單一 pivot 位置 q	等值區 [lt,gt)
遞迴範圍	A[p..q−1]、A[q+1..r]	A[l:lt]、A[gt:r]
等值鍵	除 pivot 外仍可能遞迴	整個等值區退出遞迴

本章的精確公式只計 `A[j] <= pivot`，不計索引測試、交換或亂數產生。三路實作若每筆先測 < 再測 >，一次分類可能花兩次布林比較，**精確常數必須重新推導**；不能因函式同名就照用。

兩個版本在互異鍵、均勻 pivot 下皆有期望 $\Theta(n \log n)$；全相等鍵時，二路 Lomuto 仍為 $\Theta(n^2)$，三路則一次分割即結束，為 $\Theta(n)$。

四個秩的完整比較機率表

元素對	區間大小	比較機率
(1,2)、(2,3)、(3,4)	2	各1
(1,3)、(2,4)	3	各2/3
(1,4)	4	1/2

總期望=3+4/3+1/2=29/6，與2×5×H4−16=29/6一致。兩端1、4若先遇pivot2或3就分離，先遇1或4才比較；四種首選各1/4，所以機率1/2。

相鄰秩必比較不代表所有pair必比較。精確公式也不是每次執行的比較數，例如最壞4鍵需6次，29/6是對所有亂數選擇的平均。

延伸閱讀：CLRS 第 7 章 §7.1 的分割不變量、§7.4.2 的隨機 pivot 比較分析。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study